Final Report

IoT & Non-IoT Classifier: Distinguishing Network Traffic

School of Computer Science and Informatics, Cardiff University
Bachelor of Science

Author: Harry Shepherd 22035796
Supervisor: Charith Perera
Moderator: Theodoros Spyridopoulos
5-5-2025

1 Abstract

This project focused on designing and evaluating five machine learning classifiers to differentiate IoT and non-IoT devices based on network traffic patterns. Using a UNSW dataset derived from packet capture traces the models Random Forest, Support Vector Machine, Logistic Regression, K-Nearest Neighbours, and Naïve Bayes were developed and tested. The classifiers were assessed using overall performance, execution time, and performance under resource-constrained environments. Random Forest performed the best across the metrics having the highest accuracy at 99.71% with a low execution time and a consistent performance across the resource constrained devices making it the most suitable for real-time IoT detection. These findings are important they support the deploying of machine learning-based classification in network security. Moreover, they provide insights into optimizing model selection for security and efficiency in network traffic analysis.

2 Acknowledgements

I would like to thank my supervisor Charith Perera for his guidance and feedback. Without him this project would not have been possible.

Thank you to Naeima Hamed for her guidance which was very helpful.

Finally, I would like to thank Cardiff University for the opportunity to obtain a degree

3 Table of Contents

Contents

1 Abstract i

2 Acknowledgements i

3 Table of Contents ii

4 Introduction 1

5 Background and Related Works 1

5.1 Traditional Classification Models 1

5.2 Digital Fingerprinting 2

5.3 Model Evaluation 4

6 Methodology 5

6.1 Objectives 5

Research Methodology 5

6.2 Dataset 5

6.3 Identifying Features 6

6.4 Classifier Selection 7

6.5 Evaluation metrics 8

6.5 Ethical considerations 9

7 Implementation 10

7.1 Environment and tools 10

7.2 Data preprocessing 10

7.3 Feature Matrix 12

7.4 Model Design 13

7.5 Evaluation Metrics 15

8 Results and Evaluation 18

8.1 Random Forest 18

8.2 Naïve bayes 18

8.3 KNN 19

8.4 Logistic Regression 20

8.5 SVM 21

8.6 Limitations 21

9 Future Work 22
10 Conclusion 22
11 Reflection on learning 23
12 Table of Acronyms 24
13 Appendix..... 24
14 Bibliography 35

4 Introduction

The rapid expansion of the Internet of Things (IoT) has transformed modern digital ecosystems, enabling interconnected devices to collect, exchange, and process data in real-time. IoT devices range from smart home assistants, security cameras, and healthcare monitors to industrial sensors and automated manufacturing systems. This integration of IoT into daily life has led to increased efficiency, convenience, and innovation across multiple sectors. However, as IoT adoption grows, so do concerns regarding network security, device classification, and data protection. The need for effective identification and classification of IoT devices is now crucial to prevent future cyber threats. Effective network classification is essential for identifying unknown devices and ensuring secure network management.

This project aims to implement a few different machine learning classification techniques that were found in various respected journals. The classification models will be chosen so that a wide range of techniques are represented in the evaluation.

These classification techniques will be evaluated on a several different characteristics. The aim of the evaluation is to find which classifier is the best overall, which is the best on resource constrained devices, and which is the best which technique is better for real-time classification among others. This research is significant as we will be able to tell which the best overall network classifier that is available today helping network administrators enforce security policies more effectively.

5 Background and Related Works

With the proliferation of IoT devices across the computer science industry, the need to be able to tell IoT data apart from non-IoT data is becoming vital. This look into the background will provide important context on the history of the development of IoT network classifiers and AI in this field.

5.1 Traditional Classification Models

Due to the widespread use of IoT devices, network classification is crucial for security. As Tahaei et al. [2] notes “Network traffic classification is an essential prerequisite for various network applications such as security, monitoring, accounting etc”. Because of this importance, research was conducted and five different machine learning models that were noted for high performance across multiple metrics were found.

One commonly used model is Naïve bayes which is a probabilistic classifier based on Bayes' Theorem that assumes that all input features are independent of each other. This Model was used in several papers and proved to be very time effective model having the

lowest execution time in multiple papers [9, 13]. This quick execution and simple set up show it is a solid model.

Another very popular model was K-Nearest Neighbours (KNN). KNN works by finding the 'k' closest data points to a given input and making predictions based on the majority class or the average. KNN is known to be intuitive and works well with smaller datasets but can be computationally expensive for large ones. KNN is used by the papers researched and is quite accurate with Precision, recall and F1-Score measuring 83% in Vakili et al. [13] it was also the second quickest to execute in the same paper showing that it is both an efficient form of classification and time effective which is rare for a classification system.

One classification system that was used extensively in the papers was Random Forest. Random Forest is an ensemble learning method that builds multiple decision trees during training and outputs the class that is the majority vote of the trees. It improves accuracy and reduces overfitting by using random subsets of data and features for each tree. Jmila et al. [11] noted that it was “a popular classifier due to its accuracy and speed”, but it also noted the model’s slow execution time ssaying,“its classification time grows linearly with the number of classes”. While Santos at al. [3] used random forest as the basis of its machine learning model.

Support Vector Machine (SVM) is a machine learning model that finds the optimal hyperplane that best separates different classes by maximising the margin between them. This very technical model is used in a good portion of the papers as it is a staple in network classification. Khedkar et al. [5] and Vakili et al. [13] both used SVM and they paint it as an accurate system with 78% accuracy and 98% accuracy proving it is a solid method however in both papers the Model has a high execution time showing there was a price for the high accuracy.

The final machine learning model highlighted by the background research was Logistic regression. Logistic Regression (LR) is a supervised learning algorithm used for binary classification problems. It models the probability that an input belongs to a certain class using the sigmoid function to output values between 0 and 1. The wide survey by Vakili et al. [13] had LR at mid table average of 80% accuracy but the Model had one of the quickest execution times. Cvitić et al. [12] uses an adapted version of LR to great success gaining an accuracy score of 99.79% in their dataset showing the strength of the model.

5.2 Digital Fingerprinting

Digital fingerprinting in network traffic involves extracting behavioural features that uniquely characterise devices. These features can be used to classify traffic sources, detect anomalies, or differentiate IoT from non-IoT devices.

There are a few identifying features that can be taken from the network packets sent from the devices themselves. One being the size of the packets being sent with Jmila et al. [11] saying it is one of the "most important header features" as some devices will send similar sizes of packets every time.

Another feature is the inter packet arrival time (IAT), which is the time between the end of one packet and the start of another. This is used in several papers with Jmila et al. [11] saying "one of the most useful time-related features" This is because the IAT will vary by device depending on what Hardware it has.

The transport Protocol being used was also an identifying feature for fingerprinting. Sivanathan et al. [1] identified that different IOT device use different transport protocols like Transmission Control Protocol (TCP), User Datagram Protocol (UDP) or even ICMP and IGMP. This makes Transport protocol information seemingly very important to process out of the dataset.

The Destination IP and Destination ports are another feature of packet level network data that can identify the device of the packets. Ports identify specific services on a host so IoT devices will require different services and will go to different ports and IPs than Non-IoT devices. Sivanathan et al. [1] talked about how IoT devices will have significantly fewer destination ports over one thousand packets than non-IoT devices which makes sense as laptops and pcs will perform much for duties than most IoT devices and as a result will have more destinations. This feature can have potential spoofing issues [11] but otherwise is a very solid feature to use.

In digital fingerprinting, identifying features are extracted from multiple network packets and organised into a feature matrix that can be efficiently processed by machine learning models. Although features would obviously play a big part in the feature matrix, the volume of packet data used for classification also plays a critical role. Jmila et al. [11] says there can be a huge difference between a small number of packets and a large number as large numbers can bring greater accuracy however they are usually "time and memory-consuming to analyse" [11]. There is a wide range of packet numbers across the papers with Lopez-Martin et al. [8] claiming that between 5 and 15 packets are enough for great detection results however more data will only increase accuracy.

There are two main types of reading the datasets that are used by the papers. There is continuous and iterative or batch loading. Continuous machine learning models will read the dataset in real time and make decisions using a sliding window. All of the papers researched used the iterative method as using a pre-trained model will always lead to greater accuracy however Jmila et al. [11] did note "when a new device type is added to the network or the behaviour of existing device types legitimately changes the entire model should be re-trained for all classes" which would be very time consuming.

The other method of reading information of off datasets is the iterative method. This is where the machine learning model has an already completed dataset and then partition it 80% to 20% training data to testing data. The method is effective for IoT Network classifiers as Vakili et al. [13] uses this batch loading method and it has accurate results with multiple accuracy readings above 90%. Hameed et al. [6] also uses this method to success obtaining accuracy results in the 90s as well. A lot of the papers use iterative or batch loading methods as training the models beforehand is a much more accurate system the using live training.

5.3 Model Evaluation

A good evaluation system is very important for discovering which machine learning system is the most successful in different situations. The main system that is used heavily in all machine learning system is the Confusion Matrix system. This system used in the papers [2, 3, 5, 6, 8, 9, 13], with Vakili et al. [13] saying its "one of the most intuitive and descriptive metrics used to find the accuracy and correctness of a machine learning algorithm". This simple but solid evaluation metric provides a clear breakdown of precision, recall, accuracy, and the F1 score which can be helpful in in imbalanced datasets where accuracy alone may be misleading.

It is also worth noting that some papers evaluated their models against multiple datasets like Antunes et al. [10] which evaluated against the Miller–Charles dataset and then created their own. Papers [13, 2, 6] also use multiple datasets as this clearly leads to a more robust model that can handle a variety of devices. It is unlikely that there will be time to create a dataset so one premade dataset will be sufficient.

One Evaluation technique that has largely been overlooked in the research papers that were read is evaluating the models on resource constrained devices. Jmila et al. [11] correctly identified IoT network classification as "time-consuming and computationally exhausting, requiring large storage and processing resources", and because of this the models are usually run on computer with a good amount of processing power and memory. However not everyone has these sorts of devices at hand, so it is important to know how the models perform on all types of computers.

Another very important method of evaluating the models was by their execution time. Timing how long the models take to perform their classification is a very important metric that was used by most if not all the papers researched. Execution time becomes especially critical in real-world IoT environments where devices often operate under strict time constraints and limited computational resources. Panda et al. [9] highlights how important execution time is as an evaluation metric as the quickest model, naïve bayes, was one hundred times quicker than the slowest model, random forest.

6 Methodology

6.1 Objectives

The primary aim of this project is to implement and evaluate five distinct IoT network classification techniques capable of distinguishing IoT devices from non-IoT devices based on packet data transmitted over the network. The first objective is to "find and develop five classification techniques that will classify the network packets into IoT or non-IoT.". Selecting five classifiers ensures a diverse range of models, enabling a more comprehensive evaluation and comparison of their effectiveness. The classifiers listed below have been chosen based on the research conducted in the background demonstrating their effectiveness and relevance.

The second objective is to, "collect the training IoT and non-IoT data needed for the five classification techniques." This objective is to ensure that the supervised machine learning models have good reliable data to work off to ensure the accuracy of the models.

The third object is to, "develop an evaluation system for the five classification techniques being used." This objective is to ensure a well-rounded assessment of the accuracy of the classifiers and of each model's practical viability in real world IoT environments, where constraints like memory and processing power are common.

Research Methodology

This study employed a quantitative, experimental research methodology. The classification models were developed and tested using the previously mentioned network packet dataset, with performance evaluated through three measurable metrics outlined below. To ensure fair comparison and reproducibility, the experimental design applied consistent preprocessing steps and tested multiple models under standardised conditions including a preset random state of 42. This methodology was selected to provide objective, data driven insights into the effectiveness of various classifiers for IoT device detection.

6.2 Dataset

For this project, a dataset of 31 IoT and Non-IoT devices was used and produced by UNSW Sydney for Sivanathan et al. [1]. This dataset contained Twenty days of network traffic traces collected from the devices shown in Figure 1. The CSV download of the dataset was used, which came with the data pre-processed into these groups, Packet ID, TIME, Size, eth.src (ethernet source), eth.dst (ethernet destination), IP.src (IP source), IP.dst (IP destination), IP.proto (IP Protocol), port.src (Port source), and port.dst (Port destination).

This dataset was chosen for a few reasons, the first of which is it is a relatively recent paper, published in 2016, it captures modern network traffic patterns and protocols, ensuring relevance to contemporary analysis. Another reason is the fact that it is used by several Network Classifier papers [1, 6, 11] with Jmila et al. [11] saying the dataset "has been extensively used in the literature". The dataset also contains a relatively balanced ratio of IoT Devices to non-IoT devices with eight Non-IoT devices to 22 IoT devices which ensures class balance reducing the risk of bias. The final reason for selecting the UNSW dataset for this project is its diverse range of devices. It includes 22 IoT devices, encompassing event-based devices like the smart plug, continuous devices like the Blood Pressure meter, and other categories. Additionally, the dataset features a variety of non-IoT devices, including Windows based laptops, Mac computers, as well as Android and iPhone devices. This diversity enhances the generalisability of the trained models, ensuring robust performance across different device types.

This dataset Is freely available online on UNSW Sydneys website

6.3 Identifying Features

For the classification techniques to effectively determine which packets correspond to specific devices, it is necessary to construct a feature matrix that incorporates both distinctive identifying features and the quantity of packets analysed. Therefore, a list of identifying features had to be processed from the packet data in the dataset.

The first feature considered was the transport protocol ratio, specifically the proportion of UDP to TCP. Laptops and desktops serve a wide range of functions, which necessitates the use of various transport protocols to meet these diverse requirements. In contrast, IoT devices like the Withings smart baby monitor typically have specialised purposes, such as sending video, and therefore primarily rely on UDP. As you can see from the table shown below of three IoT devices compared to the non-IoT Laptop, there is a significant difference in the Transport protocol use between the two. The Laptops TCP ratio is much lower than the other devices due to its diverse number of services.

Transport Protocol Ratio	**Laptop**	**TP-Link Smart plug**	**Withings Smart Baby Monitor**	**HP Printer**
TCP Ratio	0.276	0.912	0.985	0.948

The next feature included in the feature matrix is the size of the packets being transmitted. Different devices naturally send varying types of data, encapsulated in packets of different sizes, depending on the protocols used. Consequently, recording the minimum, maximum, and average packet sizes provides valuable insights and enhances the effectiveness of the classification process. Similarly, the total number of packets transmitted by each device throughout the day was recorded. This is because

different devices exhibit varying levels of activity; for instance, the smart plug operates on an event-based system, while devices designed for video recording are significantly more active due to their continuous data transmission.

The table below illustrates that each device exhibits a distinct pattern in transmitting network packets. The laptop, for instance, tends to have both larger average packet sizes and a wider range compared to the IoT devices. Some IoT devices, such as the baby monitor, generate notably small average packet sizes, with values as low as 80.

Packet Size	**Laptop**	**TP-Link Smart plug**	**Withings Smart Baby Monitor**	**HP Printer**
Mean size	226.502	132.553	80.898	140.85
Maximum size	1514	620	1514	1514
Minimum size	44	66	66	54

The final feature identified was the destination port and IP address. Each device served distinct functions, resulting in data being transmitted to different destinations using various ports. While laptops and desktops performed a wide range of tasks, leading to a lengthy list of IP addresses and ports, specialised devices typically communicated with only one server, reflecting their focused functionality.

The table below shows the Unique Destination IPs and ports taken from the first 1000 packets of some devices recorded in the later feature matrix dataset. As you can see the number varied greatly between devices as they have different functions and so have different servers and websites to send their data to.

Ips and Ports	**Laptop**	**TP-Link Smart plug**	**Withings Smart Baby Monitor**	**HP Printer**
Unique destination Ips	162	21	3	4
Unique destination Ports	114	4	3	5

The Feature matrix will analyse the identifiable features against 1000 packets at a time. The Research paper Bendiab et al. [15] contained a Machine learning IoT Malware detection system that worked on 1000 packet image windows, and it achieved accuracy results above 90% showing the effectiveness of using this many packets.

6.4 Classifier Selection

The five classifiers chosen for this project are Random Forest, Naïve Bayes, Logistical Regression, SVM, and KNN. The first Classifier, Random Forest, was chosen because of

its high accuracy in the research Santos et al. [3] and Sivanathan et al. [14] showing it is a very effective classifier that deserved a place. The second Classifier Naïve Bayes was chosen due to its very quick execution time having the quickest result in papers Vakili et al. [13] and Panda et al. [9]. The third classifier Logistic regression was chosen because of its very high accuracy in paper Cvitić et al. [12] showing it's a good model. The fourth classifier SVM had very high accuracy in a research paper in Vakili et al. [13]. This classifier was used throughout the research papers and was shown to be a very effective machine learning model. The last classifier was KNN. This classifier is simple but effective, correctly guessing 303 anomalies in paper Vakili et al. [13] showing its accuracy. These classifiers were chosen to represent a balanced mix of performance, and computational efficiency, providing a comprehensive comparison across different machine learning approaches to IoT device classification.

6.5 Evaluation metrics

I was judging the classifiers based on which was the best overall, which was the most time efficient and which was the best on resource constrained devices.

To determine the overall best-performing classifier, a confusion matrix to compute the true positives, false positives, true negatives, and false negatives for each classifier was utilised. Using these values, the precision, recall, F1 score, and accuracy were calculated, providing a comprehensive assessment of each classifier's performance. These values are crucial because, with network classifiers, it is essential that IoT devices are accurately identified as such. As highlighted in paper Tahaei et al. [2], IoT devices pose a poor degree of security, making them frequent targets in cyberattacks such as man-in-the-middle or pivot attacks, which can compromise entire networks. Therefore, achieving high recall in classification is vital. Misclassifying IoT devices as non-IoT poses significant risks, as it could leave these vulnerable devices unmonitored, increasing the likelihood of security breaches.

To identify the most time-efficient classifier, the Python time library was utilised to measure the prediction time for each classifier. This metric is crucial because, while accurate device classification is important, it's practical value diminishes if the process is excessively time-consuming.

The final evaluation metric focuses on determining which classifier is most suitable for resource-constrained devices. This metric is essential because not all users implementing these classifiers will have access to the high-performance computers available at the university. To address this metric, the VirtualBox virtual machine program was utilised to simulate three resource-constrained devices. These devices were configured to be nearly identical, with the primary distinction being variations in RAM allocation. This approach ensured consistency across the machines, providing a controlled environment that enhanced the reliability of the experiment. The three

computers will be configured with 2GB, 3GB, and 4GB of RAM. These values were chosen to emulate progressively constrained resource environments, representative of edge computing and small network appliances. These values serve as a structured stress test under commonly available low to mid-range hardware conditions. By beginning below standard virtual machine defaults and incrementally increasing the allocation, the objective was to pinpoint when model performance begins to decline, providing insights for deployment decisions in resource limited settings. Lower Ram values were going to be used but it was found that lower than 2GB and the OS system struggled to run Python.

6.5 Ethical considerations

When using IoT, ethical considerations should always be a top priority due to the sensitive nature of the data and the widespread impact of these technologies. The key concern is data privacy, as IoT devices often collect personal information, making it essential to ensure secure storage and proper handling of this data. The dataset used was created by a respected institution that created and handled this data in an ethical way.

Another potential concern is data bias. If some device types are overrepresented, classifiers may generalise poorly. However, the dataset used includes twenty days of traffic from a diverse set of devices, ensuring that all devices are fairly represented.

7 Implementation

7.1 Environment and tools

This project's code was implemented using Jupyter Notebook and Python 3.13.2. Python was chosen for its simplicity and robust support for machine learning and data processing libraries utilised in this project. Jupyter Notebook was selected for its modularity, enabling the execution of different classifiers simultaneously while keeping preprocessing code separate for clarity.

The key Python libraries used in this project were Pandas 2.2.3 and Scikit-learn 1.6.1. Pandas, a powerful open-source library for data manipulation, played a significant role in the preprocessing phase of the project. Scikit-learn, another open-source library, provided access to the machine learning models essential for this project. Both libraries are reliable, widely used, and well-suited for tasks of this nature.

7.2 Data preprocessing

The UNSW website contained twenty days of traffic traces but they were in separate csv downloads each of which was too small to support any reliable machine learning model. Different days of data would need to be concatenated into one master dataset for the project to begin. To do this a Python file was made that used panda 2.2.3 to join the different days together. The python file shown below concatenated two different days into one file and then saved this to a new file. All twenty days of traffic traces were combined using this script.

CSV Combiner Script

```
import pandas as pd

# Load CSV files
df1 = pd.read_csv("C:\\Users\\harry\\CombinedCSV.csv")
df2 = pd.read_csv("C:\\Users\\harry\\OneDrive\\Desktop\\New Dataset with blackjack and hookers\\16-10-12.csv\\16-10-12.csv")

combined = pd.concat([df1, df2], ignore_index=True)

# Save new file
combined.to_csv('CombinedCSV.csv', index=False)
```

Now twenty days were in one file, they needed to have the correct PacketIDs as there were many repeating IDs. The python file below was used to reset the PacketIDs and then add new values starting at 1 and finishing at the end of the file. This file resets the IDs using Panda 2.2.3.

PacketID reset Script

```python
import pandas as pd

# Load dataset
df = pd.read_csv("C:\\Users\\harry\\CombinedCSV.csv")

# Drop existing Packet ID column
df.drop(columns=["Packet ID"], inplace=True)

# Reassign a new Packet ID starting from 1
df.insert(0, "Packet ID", range(1, len(df) + 1))

# Save dataset
df.to_csv("iot_traffic_cleanedV2.csv", index=False)

print(df.head())
```

Finally now that the dataset had all twenty days with the correct PacketIDs, it needed to have IoT and non-IoT labels. As you can see from the python script below, a dictionary was created that saved the source mac addresses to the device IoT or non-IoT labels. This was used to create a new column called device_type that saved the IoT label.

IoT label Script

```python
import pandas as pd

iot_devices = {
    "d0:52:a8:00:67:5e": "IoT",
    "44:65:0d:56:cc:d3": "IoT",
    "70:ee:50:18:34:43": "IoT",
    "f4:f2:6d:93:51:f1": "IoT",
    "00:16:6c:ab:6b:88": "IoT",
    "30:8c:fb:2f:e4:b2": "IoT",
    "00:62:6e:51:27:2e": "IoT",
    "e8:ab:fa:19:de:4f": "IoT",
    "00:24:e4:11:18:a8": "IoT",
    "ec:1a:59:79:f4:89": "IoT",
    "50:c7:bf:00:56:39": "IoT",
    "74:c6:3b:29:d7:1d": "IoT",
    "ec:1a:59:83:28:11": "IoT",
    "18:b4:30:25:be:e4": "IoT",
    "70:ee:50:03:b8:ac": "IoT",
    "00:24:e4:1b:6f:96": "IoT",
    "74:6a:89:00:2e:25": "IoT",
    "00:24:e4:20:28:c6": "IoT",
    "d0:73:d5:01:83:08": "IoT",
    "18:b7:9e:02:20:44": "IoT",
    "e0:76:d0:33:bb:85": "IoT",
    "70:5a:0f:e4:9b:c0": "IoT",
    "08:21:ef:3b:fc:e3": "Non-IoT",
    "30:8c:fb:b6:ea:45": "IoT",
    "40:f3:08:ff:1e:da": "Non-IoT",
    "74:2f:68:81:69:42": "Non-IoT",
    "ac:bc:32:d4:6f:2f": "Non-IoT",
    "b4:ce:f6:a7:a3:c2": "Non-IoT",
    "d0:a6:37:df:a1:e1": "Non-IoT",
    "f4:5c:89:93:cc:85": "Non-IoT",
    "14:cc:20:51:33:ea": "Non-IoT"}

# Load the CSV dataset
df = pd.read_csv("C:\\Users\\harry\\iot_traffic_cleanedV2.csv")

# Add a new column 'device_type' by mapping MAC addresses
df["device_type"] = df["eth.src"].map(iot_devices)

df["device_type"].fillna("Unknown", inplace=True)

df.to_csv("iot_traffic_with_labelsV3.csv", index=False)
```

7.3 Feature Matrix

The CSV Dataset taken from the UNSW Journal had packet information sorted into these titles, Packet ID, TIME, Size, eth.src (ethernet source), eth.dst (ethernet destination), IP.src (IP source), IP.dst (IP destination), IP.proto (IP Protocol), port.src (Port source), and port.dst (Port destination). This information needed to be processed into the four identifiable features as mention before. To process this information off of the dataset a df.groupby(["eth.src", "packet_window"]).agg(...) call was used to group the features by source mac address and packet number.

Feature Aggregation

```
packet_window_features = df.groupby(["eth.src", "packet_window"]).agg({
    "Size": ["sum", "mean", "max", "min"],  # Traffic Volume
    "IP.proto": lambda x: (x == 6).mean(),  # TCP Ratio
    "IP.proto": lambda x: (x == 17).mean(), # UDP Ratio
    "port.dst": "nunique",  # Unique destination ports
    "IP.dst": "nunique",  # Unique destination IPs
    "device_type": lambda x: x.value_counts().idxmax()  # device type
}).reset_index()
```

The first of these features was the transport protocol ratio. This feature would be two ratios of the main two transport protocols UDP and TCP. To create these ratios the "IP.proto" column was checked for when it was either 6 for TCP or 17 for UDP. The mean of both protocols was then found to give both ratios.

For the next features were the total size of the packets being sent as well as their individual sizes. For this, a simple Sum, mean, max, and min aggregation command was used to give detailed information about the packet sizes for all devices.

The final two identifiable features aggregated off of the dataset were the unique destination IP address and destination ports. These two were also relatively simple to obtain through a simple "nunique" command of off their respective columns which collected the number of unique Ips and ports for each device for every 1000s packets.

When completed this aggregation gets saved to a "iot_1000_packet_windows" csv which is shown below in figure 11. This csv file contains the identifiable features of a source mac address averaged over 1000 packets.

7.4 Model Design

To effectively classify IoT and non-IoT devices, each machine learning model was carefully constructed using a consistent set of training procedures to ensure fair and meaningful comparison across classifiers. Some of these procedures were the same across all five of the classifiers. The procedures are shown below in Random forests code in figure 2.

The first step that all the classifiers performed was opening the 1000 packets csv file that had been created in the previous step. The features were then saved as the variable x so that the models knew what behaviour to look out for. The device type was saved as y so that the model knew what to learn to predict and so that the device type column was not in the final testing data.

Classifier loading procedure

```python
# Load the dataset
df = pd.read_csv("iot_1000_packet_windows.csv")

# Select features and target variable
features = ["total_bytes", "avg_packet_size", "max_packet_size", "min_packet_size",
            "tcp_ratio", "unique_dest_ports", "unique_dest_ips"]
X = df[features]
y = df["device_type"]
```

Once the features dataset had been loaded in, the device type column had to be transformed into binary using a label encoder so that the labels were understood by the model as of course, digital models do not understand written labels.

All the models were evaluated on a fair 80/20 training to testing data ratio. This is an industry standard and ensures that the models are trained on a majority of the data ensuring effective results on the remaining unseen data. To ensure reproducibility of my results, a random state of 42 was used so the results stay consistent every time they are run.

The final procedure that all classifiers needed was feature scaling. This is the process of normalising the range of the features so that all features are treated fairly by the classifiers. Without this, the classifiers could look at variables like the total byte size more than the other features purely because it is the largest number. To implement this Sklearn's function "StandardScaler()" was used to scale all testing and training data correctly.

The first classifier introduced was Random Forest, an ensemble method that operates by constructing a collection of decision trees. For this study, the Random Forest model was configured to use 100 decision trees to evaluate the dataset. This number strikes a good balance between achieving high accuracy, which improves with more trees, and maintaining low execution time, as fewer trees reduce computational demands. Random forests code is figure 2.

The second classifier implemented was Naïve Bayes, a probabilistic model based on Bayes' Theorem that assumes feature independence. Gaussian Naïve Bayes, used in this study, works effectively with normally distributed data. To enhance its performance, all preprocessing steps, including feature scaling, were applied. Scaling ensures the algorithm can interpret the normalised data more accurately, aligning with the assumptions of a Gaussian distribution. Naïve Bayes had no extra parameters to fine tune. Figure 3 is Naïve bayes code.

The third classifier implemented was KNN, a straightforward, instance-based algorithm that classifies data points by majority voting among their closest neighbours in the feature space. For this study, the number of neighbours was optimised to three, as this

configuration achieved excellent accuracy. All necessary preprocessing steps were applied to ensure the classifier performed effectively. Figure 4 is KNNs code.

The fourth classifier implemented was Logistic Regression, a statistical model that predicts categorical outcomes by estimating probabilities with a logistic function. The max_iter parameter, set to 1000, controlled the maximum iterations the solver could run during optimisation, balancing accurate labelling of the dataset with efficient execution time. The classifier was run with all the other preprocessing functions as the others and Logistic Regressions code is figure 5.

The final classifier implemented was SVM, a supervised learning algorithm designed to find the optimal hyperplane that separates data points into distinct classes. The parameter "kernel" determines how SVM maps the data into higher dimensions for separation. Available kernel options include polynomial, linear, radial basis function, and sigmoid. The radial basis function kernel was selected for this study as it achieved the highest accuracy, particularly due to the largely non-linear nature of the features. Figure 6 illustrates the implementation of the SVMs code.

7.5 Evaluation Metrics

As mentioned above the classifiers were judged off three main evaluation metrics, the first of which was the confusion matrix. This is a method that uses false and true positives and negatives to get a detailed understanding of a model's performance, allowing the calculation of accuracy, precision, recall, and F1 score, each offering insight into different aspects of the classifier's effectiveness.

Sklearn came with a very easy to use function that sets up a confusion matrix that displays the Precision, recall, F1-score and Accuracy of each model. This helped to find the best model overall.

Confusion matrix function
`print(f"\n SVM Model Accuracy: {accuracy_score(y_test, y_pred_svm):.2%}")`
Confusion matrix display

```
◆ SVM Model Accuracy: 97.34%
◆ Classification Report (SVM):
              precision    recall  f1-score   support

           0       0.94      0.97      0.96      1256
           1       0.99      0.98      0.98      2947

    accuracy                           0.97      4203
   macro avg       0.97      0.97      0.97      4203
weighted avg       0.97      0.97      0.97      4203
```

A Heatmap was also added using Seaborn and matplotlib that allows the user to see the raw confusion matrix for each of the classifiers including their True positives and negatives and their False Positives and negatives. This heatmap is shown with the results below.

The next evaluation metric was the most time efficient classifier. Since classifiers are often deployed in real time environments, it is crucial that they operate quickly and efficiently to ensure timely decision making and responsiveness. The python Library time contained functions that measure the time execution of the different models.

```
start = time.time()
y_pred_svm = svm_model.predict(X_test_scaled)
end = time.time()
```

```
Prediction time: 0.9200229644775391 seconds

◆ SVM Model Accuracy: 97.34%
```

The time function efficiently recorded the prediction time for each model, making it straightforward to identify which classifier performed fastest in terms of execution speed.

The third evaluation metric presented greater complexity compared to the others, focusing on determining which model performs best on resource constrained devices. To evaluate this, the models were tested within VirtualBox across three virtual machines. These computers had nearly identical specifications in terms of CPU and GPU as confirmed through terminal commands with the computers only having 1 CPU core and integrated Graphics. They differed in RAM allocations, set to 2.5GB, 3GB, and 4GB to give a range of resource constrained devices. These computes were tested on all

five of the Classifiers however SVM would not run on the 2.5GB and 3GB computer as it caused a crash when its code was run in both computers.

The virtual machines ran on the Unix-based Lubuntu operating system, chosen for its ability to maintain performance even with lower RAM values. Within this environment, the code initially written in Jupyter Notebook was converted into a Python script to ensure seamless execution on the resource-constrained setups. This approach provided a controlled platform to assess the models effectively.

8 Results and Evaluation

This section presents the collected results from the five classifiers, including their accuracy, confusion matrices, and execution times. These metrics were evaluated across both the resource-constrained devices and the primary system.

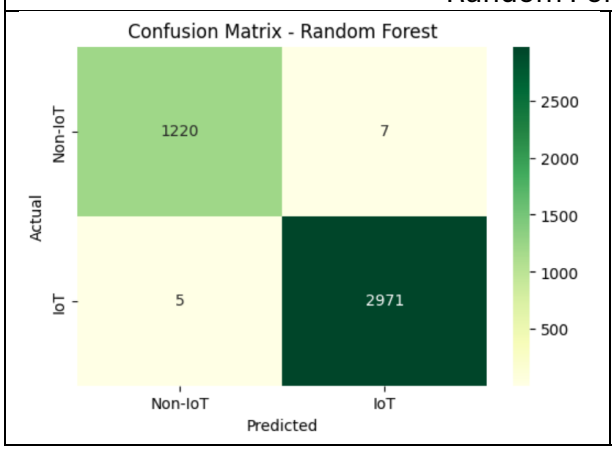
This evaluation prioritises recall, as it is essential in classification tasks especially when failing to correctly identify an IoT device incurs a significant cost. Recall is directly tied to false negatives as a lower recall results in more actual positives being misclassified, which can have serious implications for any industry system utilising these classifiers.

For the resource constrained devices, the 2.5GB, 3GB, and 4GB were shown in figure 7, 9, and 8 Respectively.

8.1 Random Forest

The first evaluation metric analysed was the confusion matrix and accuracy. Among the classifiers, Random Forest demonstrated the highest accuracy at 99.71% and achieved perfect recall at 1.00. This result is particularly significant because Random Forest not only identified nearly all IoT devices correctly but also avoided any misclassification of IoT devices as non-IoT, mitigating potential security risks. The execution time was 0.0329 Seconds, to three significant figures, which puts Random Forest third which is a very good time considering the high accuracy. For the third evaluation metric, testing on resource constrained devices, Random Forest's performance remained largely consistent, except on the 2.5GB RAM machine, where it was an order of magnitude slower, taking 0.351 seconds, to three significant figures. This suggests that, unless the system is severely resource constrained, the classifier should still perform reliably.

Random Forest Results



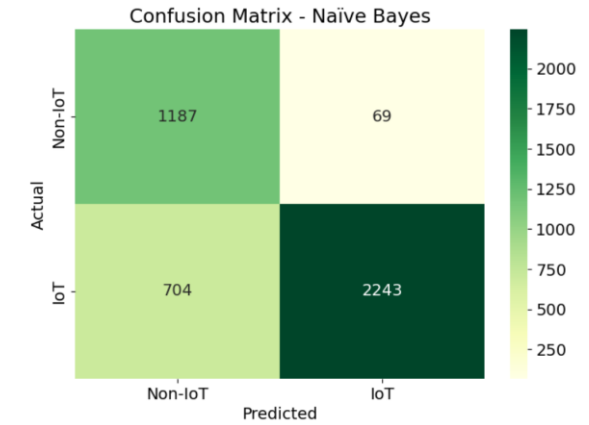

```
Prediction time: 0.0328669548034668 seconds
Model Accuracy: 99.71%
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      1227
           1       1.00      1.00      1.00      2976

    accuracy                           1.00      4203
   macro avg       1.00      1.00      1.00      4203
weighted avg       1.00      1.00      1.00      4203
```

8.2 Naïve bayes

The confusion matrix and classification report are presented below as the initial evaluation metrics. As shown, Naïve Bayes exhibited the weakest confusion matrix

performance among all classifiers, misclassifying a significant number of IoT devices as non-IoT. This compromises its reliability as a classifier and introduces potential security risks. Although it achieved a rapid prediction time of 0.00152 seconds, to three significant figures, this advantage is overshadowed by its relatively low accuracy of 81.61%, the second lowest in the project. While this accuracy aligns with findings from previous papers Sivanathan et al. [1], its poor IoT recall further diminishes its effectiveness compared to Naïve Bayes classifiers reported in other papers. When testing on resource constrained devices, the results remained largely consistent, except on the 2.5GB RAM machine, where the prediction time was slower at 0.00621 seconds, to three significant figures. This indicates that the classifier performed well in this evaluation.

Naïve Bayes Results

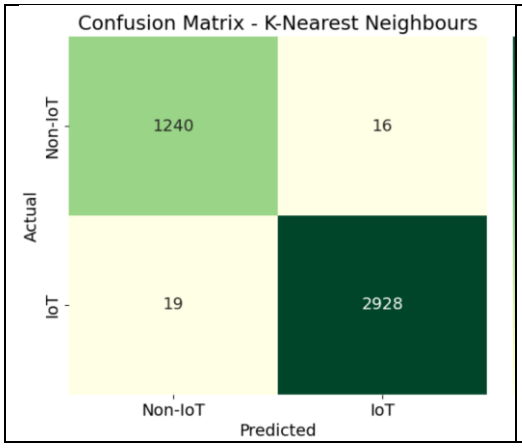



```
Prediction time: 0.0015194416046142578 seconds

Naïve Bayes Model Accuracy: 81.61%
 Classification Report (Naïve Bayes):
              precision    recall  f1-score   support

           0       0.63      0.95      0.75      1256
           1       0.97      0.76      0.85      2947

    accuracy                           0.82      4203
   macro avg       0.80      0.85      0.80      4203
weighted avg       0.87      0.82      0.82      4203
```

8.3 KNN

The confusion matrix and classification report below demonstrate KNN's strong performance as a classifier. Its recall score indicates that it accurately labelled nearly all IoT devices, and it also achieved an impressive accuracy of 99.17%, the second highest in the project. However, its prediction time was relatively slow, taking 0.245 seconds, to three significant figures, making it the second slowest classifier in the study. While this latency may render it unsuitable for real time security systems, it remains well suited for applications that do not require immediate results. For the third metric, like the other classifiers, KNN performed largely the same except a notable prediction time drop when running on the 2.5GB ram machine at 0.734 seconds, to three significant figures.

KNN Results

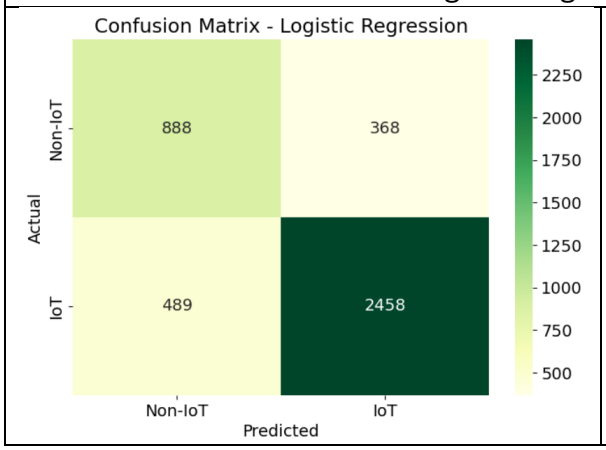



```
Prediction time: 0.24501681327819824 seconds

◆ KNN Model Accuracy: 99.17%
◆ Classification Report (KNN):
              precision    recall  f1-score

           0       0.98      0.99      0.99
           1       0.99      0.99      0.99

    accuracy                           0.99
   macro avg       0.99      0.99      0.99
weighted avg       0.99      0.99      0.99
```

8.4 Logistic Regression

Logistic Regression delivered mixed results in the evaluation. As shown in the confusion matrix and classification report, it misclassified 489 IoT devices as non-IoT, posing a significant security risk in a real-world system. Its IoT recall was the second lowest in the study, just behind Naïve Bayes, and its accuracy of 79.61% was the worst among all classifiers, making it unreliable for accurate classification. However, it excelled in prediction speed, achieving an exceptionally fast time of 0.000853 seconds (to three significant figures), outperforming all other classifiers by an order of magnitude. Despite its poor accuracy, when compared to prior studies Vakili et al. [13] where similar methods achieved accuracy as low as 50%, Logistic Regression could still be viable as a real time security classifier due to its rapid prediction time. Logistic regression performed largely the same on the resource constrained devices with its only notable change in performance being a prediction time rise on the 2.5GB of ram system at 0.00156 seconds to three significant figures.

Logistic Regressions Results



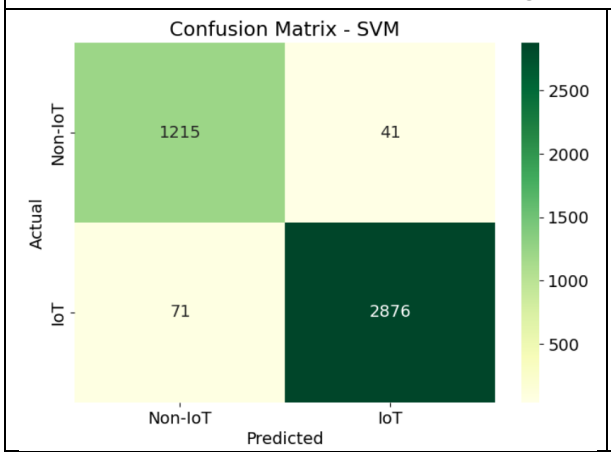

```
Prediction time: 0.0008533000946044922 seconds

◆ Logistic Regression Accuracy: 79.61%
◆ Classification Report (Logistic Regression):
              precision    recall  f1-score   support

           0       0.64      0.71      0.67      1256
           1       0.87      0.83      0.85      2947

    accuracy                           0.80      4203
   macro avg       0.76      0.77      0.76      4203
weighted avg       0.80      0.80      0.80      4203
```

8.5 SVM

This classifier demonstrated strong performance in its confusion matrix, misclassifying only 71 IoT devices and achieving a recall score of 0.98, both notable results. While its recall was not as high as other classifiers like Random Forest, it remained a highly reliable metric. However, the classifier had a significantly high prediction time, taking 0.783 seconds, to three significant figures, making it the slowest in the project. This considerable latency renders it unsuitable for real time security applications. Additionally, its accuracy was lower than models like KNN and Random Forest, both of which outperformed it while maintaining faster prediction times, making this classifier one of the less effective options.

A third evaluation criterion was its performance on resource-constrained devices. Tests were conducted on computers with 2.5GB, 3GB, and 4GB of RAM. SVM failed to run on devices with 2.5GB and 3GB of RAM, crashing upon startup, highlighting its limitations in low resource environments. Its already high prediction time may have contributed to its crashes on machines where other classifiers ran successfully. On the 4GB RAM system, its prediction time remained consistent at 0.296 seconds, to three significant figures. This further underscores the classifier's substantial resource demands, making it less suitable for deployment on constrained systems.

SVM Results




```
Prediction time: 0.7829654216766357 seconds

◆ SVM Model Accuracy: 97.34%
◆ Classification Report (SVM):
              precision    recall  f1-score   support

           0       0.94      0.97      0.96      1256
           1       0.99      0.98      0.98      2947

    accuracy                           0.97      4203
   macro avg       0.97      0.97      0.97      4203
weighted avg       0.97      0.97      0.97      4203
```

8.6 Limitations

There were some limitations in the results of this project, particularly in the evaluation of the classifiers on resource-constrained devices. The third metric, assessing performance across different machines, showed minimal variation in results besides a drop in prediction time outside of SVM. This could indicate that the classifiers are well-optimised for these devices, or it may be due to the CPU and GPU settings of the virtual machines being configured too high, potentially masking performance differences. Another limitation of the resource-constrained device metric is that the virtual

machines relied heavily on swap space to enhance performance. This likely influenced the results in some capacity, though the exact impact remains uncertain.

9 Future Work

The first recommended step for future work would be to get a more effective system for testing the classifiers on resource constrained devices. The current system saw little change across the classifiers expect for a drop in time on the lowest setting. A system with a larger range of computers with a change in not just ram would perhaps be a more effective method to measure this metric.

An important recommendation is to train and test the classifiers on multiple datasets. Relying on a single dataset can lead to biased results and limit the generalisability of the model. Expanding the evaluation to diverse datasets would provide a more robust assessment of performance and reliability.

A final additional enhancement to the project could have been the inclusion of more classifiers. With more time, deep learning models such as CNNs, along with a broader selection of machine learning classifiers, could have been integrated to provide a more diverse and comprehensive evaluation. Deep learning models, such as CNNs, represent a rapidly advancing technology. Testing their accuracy alongside other models would have provided valuable insights into their performance and comparative effectiveness.

10 Conclusion

This project focused on developing five machine learning classifiers to differentiate between IoT and non-IoT devices based on network traffic patterns. Their performance was assessed using three critical metrics, prediction time, confusion matrix effectiveness, and behaviour under resource-constrained conditions. This evaluation provides insight into both classification accuracy and the feasibility of deploying these models in real-world scenarios with limited computational resources.

Among the five classifiers developed and tested, Random Forest emerged as the best performer. It achieved the highest accuracy at 99.71% and maintained a low prediction time of 0.0329 seconds, to three significant figures, demonstrating its reliability for high accuracy, low latency classification.

Accurately classifying IoT devices is crucial, given their inherent security vulnerabilities and potential use in cyber-attacks so this project has highlighted random forest as an effective classifier for use in these network security systems. Random Forest's high recall score at 1.00 with IoT devices is a significant advantage, as such errors could create security blind spots and undermine system defences.

11 Reflection on learning

At the outset of this project, my understanding of AI and machine learning was quite basic. While I had previously developed an image classifier as part of my second-year informatics module, my knowledge beyond that was limited. As a result, I was initially uncertain about the purpose of an IoT network classifier, particularly in identifying network packets.

Through extensive research on IoT classifiers in the literature review, I have come to understand their critical role in network security systems. Due to the relatively weak security of IoT devices compared to non-IoT devices, they are frequently exploited in network attacks, such as botnets. Consequently, identifying IoT activity on a network is essential for cybersecurity. This insight has significantly expanded my understanding of how AI can be applied in real-world security contexts.

I initially approached the evaluation of classifiers with a strong emphasis on accuracy, assuming it was the ultimate metric for assessing performance. I believed that a classifier's final accuracy was the definitive measure of its quality, with no other factor being as important. However, throughout the process of building the classifiers and designing the evaluation system, I realised that accuracy alone is insufficient for real world IoT network classification.

In practical applications, execution time is just as critical as accuracy, especially since many IoT network classifiers in industry operate in real-time environments. This insight has reshaped my perspective, and moving forward, when choosing machine learning models I will take into account all aspects of the model and not just assume that accuracy is the only good metric.

Another assumption that was challenged in this project was, I assumed that creating a dataset consisting of five IoT and five non-IoT devices would be relatively straightforward. My previous experience with open-source image datasets in the image classifier project had led me to believe that obtaining data would be similarly manageable. However, once I began the task, I quickly realised the complexity involved as each device needed to record data for approximately a week to gather a sufficient amount of packet information. This was not achievable in the time frame of the project.

This challenge forced me to reassess my approach to dataset creation. In the end, I opted to use a pre-existing dataset from UNSW, which provided the necessary data while ensuring a more efficient workflow. In the future I will approach dataset creation with a realistic understanding of the time and resource constraints involved. I now recognise the importance of balancing feasibility with research goals and will consider pre-existing datasets more strategically while exploring ways to refine or supplement them if needed.

The aims and objectives evolved over the course of the project. Initially, the first objective was “develop three classification techniques that will classify the network packets into IoT or non-IoT.”. However, this number was later increased to five to allow for a more comprehensive evaluation. Incorporating a broader range of classifiers, this being Random Forest, KNN, Logistic Regression, Support Vector Machine, and Naïve Bayes, enhanced the project's robustness by enabling meaningful comparisons across models with distinct strengths. Additionally, this adjustment helped simulate a more realistic decision making process akin to what a network engineer might encounter in industry.

Overall, this project has been highly rewarding, allowing me to significantly enhance my skills in machine learning and IoT. I had the opportunity to apply these skills to a challenging problem, deepening my understanding of IoT's role in network security and its vulnerabilities. Additionally, this experience has improved my ability to manage large-scale projects, refining my time management and organisational skills while navigating complex technical challenges. Even though I have had some problems with this project like creating datasets, I was able to learn from these challenges.

12 Table of Acronyms

Full name	Acronyms
Internet of things	IoT
K-Nearest Neighbours	KNN
Support Vector Machine	SVM
Transmission Control Protocol	TCP
User Datagram Protocol	UDP
Logistic Regression	LR

13 Appendix

Figure 1

List of Devices	MAC ADDRESS	Connection Type
Smart Things	d0:52:a8:00:67:5e	Wired
Amazon Echo	44:65:0d:56:cc:d3	Wireless
Netatmo Welcome	70:ee:50:18:34:43	Wireless
TP-Link Day Night Cloud camera	f4:f2:6d:93:51:f1	Wireless
Samsung SmartCam	00:16:6c:ab:6b:88	Wireless
Dropcam	30:8c:fb:2f:e4:b2	Wireless
Insteon Camera	00:62:6e:51:27:2e	Wired
	e8:ab:fa:19:de:4f	Wireless
Withings Smart Baby Monitor	00:24:e4:11:18:a8	Wired
Belkin Wemo switch	ec:1a:59:79:f4:89	Wireless
TP-Link Smart plug	50:c7:bf:00:56:39	Wireless
iHome	74:c6:3b:29:d7:1d	Wireless
Belkin wemo motion sensor	ec:1a:59:83:28:11	Wireless
NEST Protect smoke alarm	18:b4:30:25:be:e4	Wireless
Netatmo weather station	70:ee:50:03:b8:ac	Wireless
Withings Smart scale	00:24:e4:1b:6f:96	Wireless
Blipcare Blood Pressure meter	74:6a:89:00:2e:25	Wireless
Withings Aura smart sleep sensor	00:24:e4:20:28:c6	Wireless
Light Bulbs LiFX Smart Bulb	d0:73:d5:01:83:08	Wireless
Triby Speaker	18:b7:9e:02:20:44	Wireless
PIX-STAR Photo-frame	e0:76:d0:33:bb:85	Wireless
HP Printer	70:5a:0f:e4:9b:c0	Wireless
Samsung Galaxy Tab	08:21:ef:3b:fc:e3	Wireless
Nest Dropcam	30:8c:fb:b6:ea:45	Wireless
Android Phone	40:f3:08:ff:1e:da	Wireless
Laptop	74:2f:68:81:69:42	Wireless
MacBook	ac:bc:32:d4:6f:2f	Wireless
Android Phone	b4:ce:f6:a7:a3:c2	Wireless
IPhone	d0:a6:37:df:a1:e1	Wireless
MacBook/Iphone	f4:5c:89:93:cc:85	Wireless
TPLink Router Bridge LAN (Gateway)	14:cc:20:51:33:ea	

Figure 2

```python
# Load the dataset
df = pd.read_csv("iot_1000_packet_windows.csv")


features = [
    "total_bytes", "avg_packet_size", "max_packet_size", "min_packet_size",  "tcp_ratio", "unique_dest_ports", "unique_dest_ips"]
X = df[features]
y = df["device_type"]

# Convert Labels
le = LabelEncoder()
y = le.fit_transform(y)

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Random Forest Model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

start = time.time()

# Predict and Evaluate

y_pred_RF = rf_model.predict(X_test)

end = time.time()

# Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred_RF)

# Plot it nicely
plt.figure(figsize=(8,5.5))
plt.rcParams['font.size'] = 14
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGn', xticklabels=["Non-IoT", "IoT"], yticklabels=["Non-IoT", "IoT"])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Random Forest')
plt.show()

print("Prediction time:", end - start, "seconds")
print(f"Model Accuracy: {accuracy_score(y_test, y_pred_RF):.2%}")
print("Classification Report:\n", classification_report(y_test, y_pred_RF))
```

Figure 3

```python
# Load dataset
df = pd.read_csv("iot_1000_packet_windows.csv")

# Select features and target variable
features = ["total_bytes", "avg_packet_size", "max_packet_size", "min_packet_size",
            "tcp_ratio", "unique_dest_ports", "unique_dest_ips"]
X = df[features]
y = df["device_type"]

# Convert labels
le = LabelEncoder()
y = le.fit_transform(y)

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Model
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

# Predict

start = time.time()
y_pred_nb = nb_model.predict(X_test)
end = time.time()

#confusion matrix
cm = confusion_matrix(y_test, y_pred_nb)

# Plot
plt.figure(figsize=(8,5.5))
plt.rcParams['font.size'] = 14
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGn', xticklabels=["Non-IoT", "IoT"], yticklabels=["Non-IoT", "IoT"])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Naïve Bayes')
plt.show()


print("Prediction time:", end - start, "seconds")
print(f"\nNaïve Bayes Model Accuracy: {accuracy_score(y_test, y_pred_nb):.2%}")
print(" Classification Report (Naïve Bayes):\n", classification_report(y_test, y_pred_nb))
```

Figure 4

```python
# Load dataset
df = pd.read_csv("iot_1000_packet_windows.csv")

# Select features
features = ["total_bytes", "avg_packet_size", "max_packet_size", "min_packet_size",
            "tcp_ratio", "unique_dest_ports", "unique_dest_ips"]
X = df[features]
y = df["device_type"]

# Convert labels
le = LabelEncoder()
y = le.fit_transform(y)

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Model
knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_train_scaled, y_train)

# Predict

start = time.time()
y_pred_knn = knn_model.predict(X_test_scaled)
end = time.time()

# confusion matrix
cm = confusion_matrix(y_test, y_pred_knn)

# Plot
plt.figure(figsize=(8,5.5))
plt.rcParams['font.size'] = 14
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGn', xticklabels=["Non-IoT", "IoT"], yticklabels=["Non-IoT", "IoT"])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - K-Nearest Neighbours')
plt.show()


print("Prediction time:", end - start, "seconds")
print(f"\n ◆  KNN Model Accuracy: {accuracy_score(y_test, y_pred_knn):.2%}")
print(" ◆  Classification Report (KNN):\n", classification_report(y_test, y_pred_knn))
```

Figure 5

```python
# Load dataset
df = pd.read_csv("iot_1000_packet_windows.csv")

# Select features
features = ["total_bytes", "avg_packet_size", "max_packet_size", "min_packet_size",
            "tcp_ratio", "unique_dest_ports", "unique_dest_ips"]
X = df[features]
y = df["device_type"]

# Convert labels
le = LabelEncoder()
y = le.fit_transform(y)

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train  Model
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)

# Predict
start = time.time()
y_pred_log = log_model.predict(X_test_scaled)
end = time.time()

# confusion matrix
cm = confusion_matrix(y_test, y_pred_log)

# Plot
plt.figure(figsize=(8,5.5))
plt.rcParams['font.size'] = 14
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGn', xticklabels=["Non-IoT", "IoT"], yticklabels=["Non-IoT", "IoT"])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()


print("Prediction time:", end - start, "seconds")
print(f"\n ◆  Logistic Regression Accuracy: {accuracy_score(y_test, y_pred_log):.2%}")
print(" ◆  Classification Report (Logistic Regression):\n", classification_report(y_test, y_pred_log))
```

Figure 6

```python
# Load dataset
df = pd.read_csv("iot_1000_packet_windows.csv")

# Select features
features = ["total_bytes", "avg_packet_size", "max_packet_size", "min_packet_size",
            "tcp_ratio", "unique_dest_ports", "unique_dest_ips"]
X = df[features]
y = df["device_type"]

# Convert labels
le = LabelEncoder()
y = le.fit_transform(y)

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Model
svm_model = SVC(kernel='rbf')
svm_model.fit(X_train_scaled, y_train)

# Predict
start = time.time()
y_pred_svm = svm_model.predict(X_test_scaled)
end = time.time()

# confusion matrix
cm = confusion_matrix(y_test, y_pred_svm)

# Plot
plt.figure(figsize=(8,5.5))
plt.rcParams['font.size'] = 14
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGn', xticklabels=["Non-IoT", "IoT"], yticklabels=["Non-IoT", "IoT"])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - SVM')
plt.show()


print("Prediction time:", end - start, "seconds")

print(f"\n ◆ SVM Model Accuracy: {accuracy_score(y_test, y_pred_svm):.2%}")

print(" ◆ Classification Report (SVM):\n", classification_report(y_test, y_pred_svm))
```

2.5GB Ram Results Figure 7

```
2500

Prediction time: 0.350998001098632812 seconds
Model Accuracy: 99.71%
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      1227
           1       1.00      1.00      1.00      2976

    accuracy                           1.00      4203
   macro avg       1.00      1.00      1.00      4203
weighted avg       1.00      1.00      1.00      4203

Prediction time: 0.0062071163177490234 seconds

Naïve Bayes Model Accuracy: 81.61%
 Classification Report (Naïve Bayes):
              precision    recall  f1-score   support

           0       0.63      0.95      0.75      1256
           1       0.97      0.76      0.85      2947

    accuracy                           0.82      4203
   macro avg       0.80      0.85      0.80      4203
weighted avg       0.87      0.82      0.82      4203

Prediction time: 0.73377809524536133 seconds

KNN Model Accuracy: 99.17%
Classification Report (KNN):
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      1256
           1       0.99      0.99      0.99      2947

    accuracy                           0.99      4203
   macro avg       0.99      0.99      0.99      4203
weighted avg       0.99      0.99      0.99      4203
```

```
Prediction time: 0.00156226043701171875 seconds

Logistic Regression Accuracy: 79.61%
Classification Report (Logistic Regression):
              precision    recall  f1-score   support

           0       0.64      0.71      0.67      1256
           1       0.87      0.83      0.85      2947

    accuracy                           0.80      4203
   macro avg       0.76      0.77      0.76      4203
weighted avg       0.80      0.80      0.80      4203
```

4GB Ram Figure 8

```
Model Accuracy: 99.71%
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      1227
           1       1.00      1.00      1.00      2976

    accuracy                           1.00      4203
   macro avg       1.00      1.00      1.00      4203
weighted avg       1.00      1.00      1.00      4203

Prediction time: 0.0016703605651855469 seconds

Naïve Bayes Model Accuracy: 81.61%
 Classification Report (Naïve Bayes):
              precision    recall  f1-score   support

           0       0.63      0.95      0.75      1256
           1       0.97      0.76      0.85      2947

    accuracy                           0.82      4203
   macro avg       0.80      0.85      0.80      4203
weighted avg       0.87      0.82      0.82      4203

Prediction time: 0.13956427574157715 seconds

• KNN Model Accuracy: 99.17%
• Classification Report (KNN):
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      1256
           1       0.99      0.99      0.99      2947

    accuracy                           0.99      4203
   macro avg       0.99      0.99      0.99      4203
weighted avg       0.99      0.99      0.99      4203

Prediction time: 0.0002837181091308594 seconds

• Logistic Regression Accuracy: 79.61%
• Classification Report (Logistic Regression):
              precision    recall  f1-score   support

           0       0.64      0.71      0.67      1256
           1       0.87      0.83      0.85      2947

    accuracy                           0.80      4203
   macro avg       0.76      0.77      0.76      4203
weighted avg       0.80      0.80      0.80      4203

Prediction time: 0.0002770423889160156 seconds

• Logistic Regression Accuracy: 79.61%
• Classification Report (Logistic Regression):
              precision    recall  f1-score   support

           0       0.64      0.71      0.67      1256
           1       0.87      0.83      0.85      2947

    accuracy                           0.80      4203
   macro avg       0.76      0.77      0.76      4203
weighted avg       0.80      0.80      0.80      4203

Prediction time: 0.2956349849700928 seconds

• SVM Model Accuracy: 97.34%
• Classification Report (SVM):
              precision    recall  f1-score   support

           0       0.94      0.97      0.96      1256
           1       0.99      0.98      0.98      2947

    accuracy                           0.97      4203
   macro avg       0.97      0.97      0.97      4203
weighted avg       0.97      0.97      0.97      4203
```

3GB Ram Results Figure 9

Prediction time: 0.020895719528198242 seconds
Model Accuracy: 99.71%
Classification Report:

	precision	recall	f1-score	support
0	1.00	0.99	1.00	1227
1	1.00	1.00	1.00	2976
accuracy			1.00	4203
macro avg	1.00	1.00	1.00	4203
weighted avg	1.00	1.00	1.00	4203

Prediction time: 0.0003612041473388672 seconds

Naïve Bayes Model Accuracy: 81.61%
Classification Report (Naïve Bayes):

	precision	recall	f1-score	support
0	0.63	0.95	0.75	1256
1	0.97	0.76	0.85	2947
accuracy			0.82	4203
macro avg	0.80	0.85	0.80	4203
weighted avg	0.87	0.82	0.82	4203

Prediction time: 0.14486193656921387 seconds

- KNN Model Accuracy: 99.17%
- Classification Report (KNN):

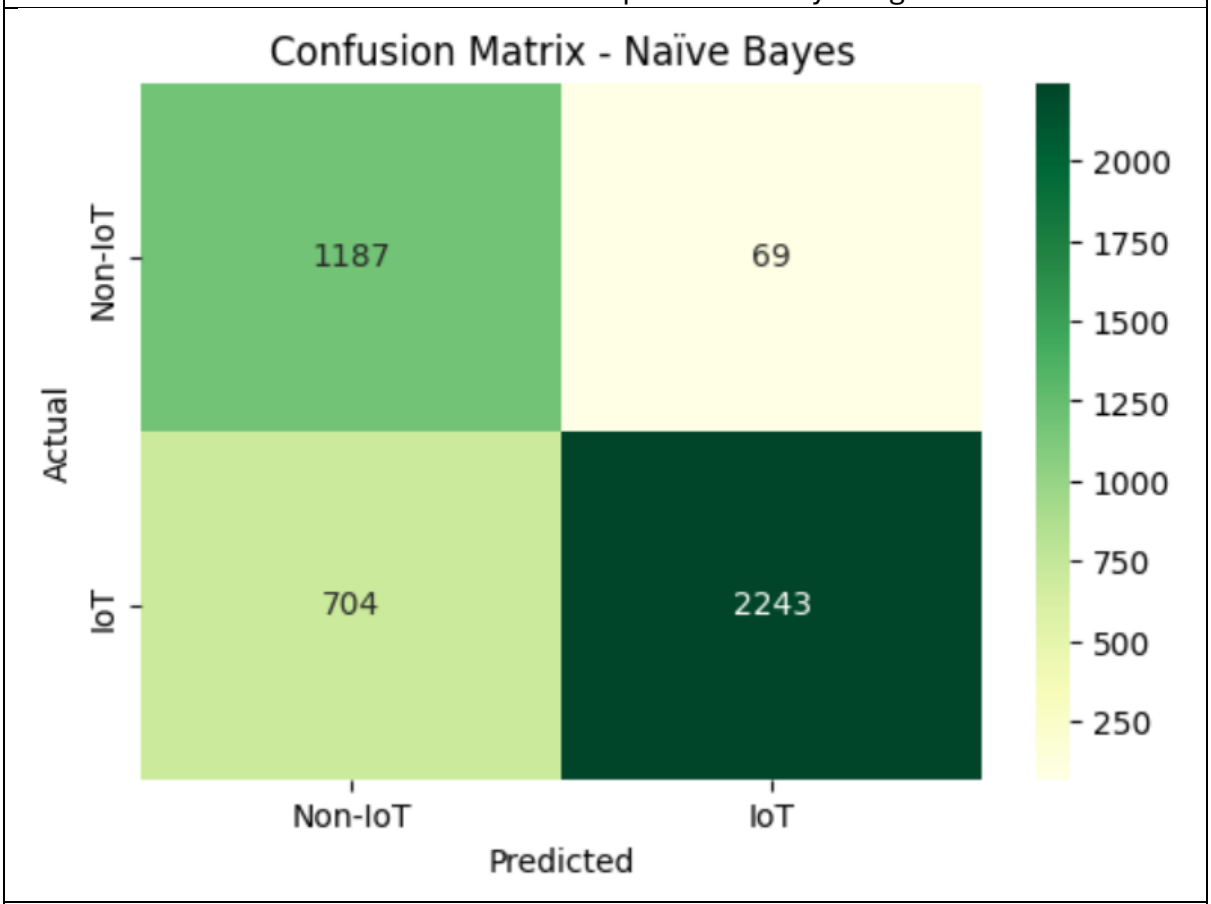
	precision	recall	f1-score	support
0	0.98	0.99	0.99	1256
1	0.99	0.99	0.99	2947
accuracy			0.99	4203
macro avg	0.99	0.99	0.99	4203
weighted avg	0.99	0.99	0.99	4203

Prediction time: 0.0002617835998535156 seconds

- Logistic Regression Accuracy: 79.61%
- Classification Report (Logistic Regression):

	precision	recall	f1-score	support
0	0.64	0.71	0.67	1256
1	0.87	0.83	0.85	2947
accuracy			0.80	4203
macro avg	0.76	0.77	0.76	4203
weighted avg	0.80	0.80	0.80	4203

The confusion Matrix Heatmap for Naïve Bayes Figure 10



1000 packet window CSV Figure 11

```
device,packet_window,total_bytes,avg_packet_size,max_packet_size,min_packet_size,tcp_ratio,unique_dest_ports,unique_dest_ips,device_type
00:16:6c:ab:6b:88,0,380111,380.111,1236,66,0.351,41,5,IoT
00:16:6c:ab:6b:88,1,376089,376.089,1236,66,0.393,48,3,IoT
00:16:6c:ab:6b:88,2,378332,378.332,1236,66,0.372,43,3,IoT
00:16:6c:ab:6b:88,3,379064,379.064,1236,66,0.372,44,3,IoT
00:16:6c:ab:6b:88,4,377745,377.745,1236,66,0.375,43,3,IoT
00:16:6c:ab:6b:88,5,373119,373.119,1236,66,0.371,43,7,IoT
00:16:6c:ab:6b:88,6,379693,379.693,1236,66,0.361,41,3,IoT
00:16:6c:ab:6b:88,7,368400,368.4,1236,66,0.39,46,7,IoT
00:16:6c:ab:6b:88,8,375194,375.194,1236,66,0.388,51,5,IoT
00:16:6c:ab:6b:88,9,374614,374.614,1236,66,0.378,46,6,IoT
00:16:6c:ab:6b:88,10,349903,349.903,1236,66,0.409,35,7,IoT
00:16:6c:ab:6b:88,11,364843,364.843,1236,66,0.377,39,9,IoT
00:16:6c:ab:6b:88,12,359934,359.934,1236,66,0.386,40,9,IoT
00:16:6c:ab:6b:88,13,372731,372.731,1236,66,0.377,44,6,IoT
00:16:6c:ab:6b:88,14,364392,364.392,1236,66,0.372,37,9,IoT
00:16:6c:ab:6b:88,15,368288,368.288,1236,66,0.38,43,8,IoT
00:16:6c:ab:6b:88,16,338107,338.107,1236,44,0.414,44,16,IoT
00:16:6c:ab:6b:88,17,328505,328.505,1236,44,0.333,43,8,IoT
00:16:6c:ab:6b:88,18,366049,366.049,1236,44,0.371,40,8,IoT
00:16:6c:ab:6b:88,19,368222,368.222,1236,66,0.378,41,8,IoT
00:16:6c:ab:6b:88,20,362100,362.1,1236,66,0.382,39,11,IoT
00:16:6c:ab:6b:88,21,340923,340.923,1236,66,0.423,41,10,IoT
00:16:6c:ab:6b:88,22,357212,357.212,1236,66,0.368,31,13,IoT
00:16:6c:ab:6b:88,23,375494,375.494,1236,66,0.378,47,6,IoT
00:16:6c:ab:6b:88,24,377075,377.075,1236,66,0.379,46,3,IoT
00:16:6c:ab:6b:88,25,379069,379.069,1236,66,0.361,44,5,IoT
00:16:6c:ab:6b:88,26,379434,379.434,1236,66,0.366,42,3,IoT
00:16:6c:ab:6b:88,27,378148,378.148,1236,66,0.374,43,3,IoT
```

14 Bibliography

[1] Sivanathan, A., Gharakheili, H.H., Loi, F., Radford, A., Wijenayake, C., Vishwanath, A. and Sivaraman, V. 2019. Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics. *IEEE Transactions on Mobile Computing* 18(8), pp. 1745–1759. Available at: https://ieeexplore.ieee.org/document/8440758/ [Accessed: 12 February 2025].

[2] Tahaei, H., Afifi, F., Asemi, A., Zaki, F. and Anuar, N.B. 2020. The rise of traffic classification in IoT networks: A survey. *Journal of Network and Computer Applications* 154, p. 102538. Available at: https://linkinghub.elsevier.com/retrieve/pii/S1084804520300126 [Accessed: 12 February 2025].

[3] Santos, M.R.P., Andrade, R.M.C., Gomes, D.G. and Callado, A.C. 2018. An efficient approach for device identification and traffic classification in IoT ecosystems. In: *2018 IEEE Symposium on Computers and Communications (ISCC)*. Natal: IEEE, pp. 00304–00309. Available at: https://ieeexplore.ieee.org/document/8538630/ [Accessed: 12 February 2025].

[4] Novel Classification of IoT Devices Based on Traffic Flow Features: 2021. *Journal of Organizational and End User Computing* 33(6), pp. 0–0. Available at: http://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/JOEUC.20211101oa12 [Accessed: 12 February 2025].

[5] Khedkar, S.P. and AroulCanessane, R. 2020. Machine Learning Model for classification of IoT Network Traffic. In: *2020 Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*. Palladam, India: IEEE, pp. 166–170. Available at: https://ieeexplore.ieee.org/document/9243468/ [Accessed: 12 February 2025].

[6] Hameed, A., Violos, J. and Leivadeas, A. 2022. A Deep Learning Approach for IoT Traffic Multi-Classification in a Smart-City Scenario. *IEEE Access* 10, pp. 21193–21210. Available at: https://ieeexplore.ieee.org/document/9718208/ [Accessed: 12 February 2025].

[7] Sivagami, P., Illavarason, P., Harikrishnan, R. and Reddy, G. 2021. IoT Ecosystem- A survey on Classification of IoT. In: *Proceedings of the Fist International Conference on Advanced Scientific Innovation in Science, Engineering and Technology, ICASISET 2020, 16-17 May 2020, Chennai, India*. Chennai, India: EAI. Available at: c [Accessed: 12 February 2025].

[8] Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A. and Lloret, J. 2017. Network Traffic Classifier With Convolutional and Recurrent Neural Networks for Internet of

Things. *IEEE Access* 5, pp. 18042–18050. Available at: https://ieeexplore.ieee.org/document/8026581/ [Accessed: 12 February 2025].

[9] Panda, S. and Panda, G. 2020. Intelligent Classification of IoT Traffic in Healthcare Using Machine Learning Techniques. In: *2020 6th International Conference on Control, Automation and Robotics (ICCAR)*. Singapore: IEEE, pp. 581–585. Available at: https://ieeexplore.ieee.org/document/9107979/ [Accessed: 12 February 2025].

[10] Antunes, M., Gomes, D. and Aguiar, R.L. 2018. Towards IoT data classification through semantic features. *Future Generation Computer Systems* 86, pp. 792–798. Available at: https://linkinghub.elsevier.com/retrieve/pii/S0167739X17327553 [Accessed: 12 February 2025].

[11] Jmila, H., Blanc, G., Shahid, M.R. and Lazrag, M. 2022. A Survey of Smart Home IoT Device Classification Using Machine Learning-Based Network Traffic Analysis. *IEEE Access* 10, pp. 97117–97141. Available at: https://ieeexplore.ieee.org/document/9881503/ [Accessed: 12 February 2025].

[12] Cvitić, I., Peraković, D., Periša, M. and Gupta, B. 2021. Ensemble machine learning approach for classification of IoT devices in smart home. *International Journal of Machine Learning and Cybernetics* 12(11), pp. 3179–3202. Available at: https://link.springer.com/10.1007/s13042-020-01241-0 [Accessed: 12 February 2025].

[13] Vakili, M., Ghamsari, M. and Rezaei, M. 2020. Performance Analysis and Comparison of Machine and Deep Learning Algorithms for IoT Data Classification. Available at: http://arxiv.org/abs/2001.09636 [Accessed: 12 February 2025].

[14] Sivanathan, A., Sherratt, D., Gharakheili, H.H., Radford, A., Wijenayake, C., Vishwanath, A. and Sivaraman, V. 2017. *Characterizing and classifying IoT traffic in smart cities and campuses*. Available at: https://ieeexplore.ieee.org/abstract/document/8116438 [Accessed: 25 April 2022].

[15] Bendiab, G., Shiaeles, S., Alruban, A. and Kolokotronis, N. 2020. *IoT Malware Network Traffic Classification using Visual Representation and Deep Learning*. Available at: https://ieeexplore.ieee.org/document/9165381 [Accessed: 9 July 2022].